Media Management System — Hosting & Deployment Options

About the Platform

The Media Management System is a digital asset management platform built for managing and distributing branded media — images, social media posts, and video reels. End users browse a media library, preview content, copy social media captions, and download assets. Authenticated editors create and manage content. Administrators manage users and system configuration.

The platform is a two-tier web application:

- **Frontend** — a Next.js application that serves the media library interface, search and filtering, card previews, and download functionality
- **Backend** — an Express.js API server that handles authentication, media processing (image resizing, video transcoding, thumbnail generation), file storage, and database operations

Both tiers are production-ready and currently deployed together via Docker Compose on a development server.

How It Works Today

The system is deployed for a single client (Zive) on a VPS running Docker. The two containers — frontend and backend — sit behind an nginx reverse proxy that handles SSL termination. Media files are stored in AWS S3. User and content data lives in a MongoDB Atlas database. The application is configured for Zive branding (logo, colors, domain, external links) via environment variables and a brand configuration file.

```
Internet → nginx (SSL) → Frontend container (port 5000)
                       → Backend container (port 5001) → MongoDB Atlas
                                                       → AWS S3
```

White-Label Brand System

The platform was designed from the outset to support multiple clients from a single codebase. A brand configuration system controls all client-facing identity:

Configurable Element	How It's Set
Company name and app title	Brand config file
Logo and favicon	Asset files in `/public/`
Header color, primary color, role badge colors	Brand config theme object
External links (portal, training)	Brand config or `null` to hide
Domain name	Environment variable
Trash retention period	Brand config
API URL, CORS origins, email sender	Environment variables

Each client deployment requires:

1. **A brand configuration file** — a TypeScript file defining the client's name, colors, logo path, and external links (~30 lines)
2. **Brand assets** — the client's logo and favicon placed in the `/public/` directory
3. **Environment variables** — database connection, S3 credentials, JWT secret, domain, CORS origins, email configuration
4. **Infrastructure** — compute, storage, and database services (the subject of this document)

No application code changes are needed to deploy for a new client. The brand system is build-time configurable — set `NEXT_PUBLIC_BRAND_CONFIG=clientname` and the frontend compiles with that client's branding baked in. The backend is brand-agnostic; it serves whatever frontend is pointed at it.

What a New Client Deployment Requires

For each new client, the work breaks down into two categories:

Brand Setup (same for all hosting options):

- Create brand config file from template (~1 hour)
- Obtain and format client logo/favicon (~30 min)
- Create S3 bucket and configure CORS/permissions (~1 hour)
- Set up MongoDB Atlas database (or equivalent) (~1 hour)
- Configure SendGrid or Mailgun for client's email domain (~30 min)
- Create DNS records for client's domain (~30 min)
- Create initial admin user and verify functionality (~30 min)

Infrastructure Setup (varies by hosting option — see below):

The hours and costs in the options table below cover infrastructure setup only. Brand setup adds approximately 4–6 hours to every option. This work is identical regardless of which hosting option is selected.

Overview

This document describes the available hosting configurations for the Media Management System. Each option is evaluated for technical complexity, cost, scope of work, and client impact.

All options below deploy the same codebase — differences are in infrastructure, not application functionality.

Runtime Resource Requirements

The backend service has specific resource requirements driven by its media processing dependencies. These requirements determine minimum viable instance sizes across all hosting options.

Backend Dependencies Driving Resource Needs

Dependency	Function	Resource Impact
Puppeteer + Chromium	Headless browser for rendering	~400MB RAM per instance; requires system libraries (GTK, NSS, X11)
Sharp	Image processing/resizing	~50MB RAM; native bindings (libvips)
FFmpeg / FFprobe	Video processing, metadata extraction	CPU-intensive during transcoding; ~100MB disk for binaries

Mongoose + MongoDB driver	Database connectivity	~50MB RAM for connection pool (10 max connections)
AWS SDK (S3)	File storage operations	Minimal footprint

Minimum Instance Sizes

Service	Minimum RAM	Recommended RAM	Minimum CPU	Notes
Frontend (Next.js)	256MB	512MB	0.5 vCPU	Standalone Node.js server; lightweight at runtime
Backend (Express)	1GB	2GB	1 vCPU	Puppeteer/Chromium is the primary driver; Sharp and FFmpeg add burst demand

Current Docker Configuration

Container	Memory Limit	Reason
`media-frontend`	512MB	Next.js standalone mode is efficient; 512MB provides headroom
`media-backend`	2GB	Puppeteer spawns Chromium processes during media operations; 2GB prevents OOM kills under load

Platform Sizing Guidance

Platform	Frontend Tier	Backend Tier	Notes
VPS (Docker)	Shared (512MB limit)	Shared (2GB limit)	4GB VPS minimum to run both containers + OS
AWS App Runner	1 vCPU / 2GB	1 vCPU / 2GB	App Runner minimum is 0.25 vCPU / 0.5GB; backend needs 2GB
Azure App Service	B1 (1 core / 1.75GB)	B2 (2 cores / 3.5GB)	B1 too small for backend; B2 provides safe margin
Render	Starter (512MB)	Standard (2GB)	Starter insufficient for backend; Standard is minimum viable
Vercel	Managed (serverless)	N/A — backend hosted elsewhere	Vercel handles frontend scaling automatically

> ***Important:*** *The backend cannot run reliably on instances smaller than 1GB RAM. Puppeteer's Chromium instance alone requires ~400MB, and under concurrent media uploads the process can spike to 1.5GB+. The 2GB recommendation includes headroom for these spikes.*

Options Summary

Option	Host (Compute)	Storage	Data	Code Changes	Hours (est–contingency)	Est. Monthly
1. VPS + Docker	Self-managed VPS	AWS S3	MongoDB Atlas	None	16–24	$13–86
2. AWS + Atlas	AWS App Runner	AWS S3	MongoDB Atlas	None	16–24	$24–108
3. Azure + Atlas	Azure App Service	AWS S3 or Azure Blob	MongoDB Atlas	None	16–24	$40–101
4a. Vercel + Render	Vercel (FE) + Render (BE)	AWS S3	MongoDB Atlas	None	16–24	$26–107
4b. Render (both)	Render (FE) + Render (BE)	AWS S3	MongoDB Atlas	None	16–24	$33–94
5. AWS Native	AWS App Runner	AWS S3	AWS DocumentDB	Auth/user refactor	48–64	$80–107
6. Azure Native	Azure App Service	Azure Blob	Azure CosmosDB	Compat fixes + S3 migration	32–48	$41–54
7. PostgreSQL Rewrite	Any of the above	Any of the above	Any PostgreSQL	Full DB layer rewrite	56–72	$14–75

> *Hours shown as base estimate through 50% contingency. Monthly costs vary by database tier (free vs. dedicated) and traffic volume.*

Option 1: VPS + Docker Compose

Architecture

```
┌──────────────────────────────────────────┐
│          VPS (4GB+ RAM)                  │
│   ┌──────────┐                           │
│   │  nginx   │ ← SSL termination         │
│   └────┬─────┘                           │
│        │                                 │
│   ┌────▼─────┐     ┌──────────────┐     │
│   │ Frontend │     │   Backend    │     │
│   │ (Next.js) │──►│  (Express)   │     │
│   │ 512MB    │     │  2GB         │     │
│   └──────────┘     └──────┬───────┘     │
└──────────────────────────┼──────────────┘
                           │
              ┌────────────▼──────────────┐
              │  MongoDB Atlas    AWS S3  │
```

```
              │  (external)      (external) │
              └─────────────────────────────┘
```

Components

Component	Service	Est. Monthly
VPS (4GB RAM)	DigitalOcean, Linode, or Hetzner	$12–24
Database	MongoDB Atlas M0 (free) or M10 (dedicated)	$0–57
File storage	AWS S3	$1–5
Email	SendGrid free tier (100 emails/day)	$0
SSL	Let's Encrypt (auto-renewed)	$0
Total		**$13–86**

Scope of Work

- Provision VPS and install Docker
- Clone repository, create `.env` with client credentials
- Configure brand settings (logo, colors, domain)
- Set up nginx reverse proxy with SSL
- Run `docker compose up --build -d`
- Create initial admin user, verify all routes
- Configure DNS records

Pros

- Lowest monthly cost
- Predictable pricing — no usage-based surprises
- Full control over infrastructure
- Single-command deployment and updates
- Application deploys as-is with zero code changes
- Both services managed as a unit via Docker Compose

Cons

- Client or provider responsible for OS updates and security patches
- No auto-scaling — fixed capacity
- Manual SSL renewal management (unless scripted)
- Server monitoring must be configured separately
- Single point of failure without additional redundancy

Client Impact

No application changes. Client receives a dedicated server running the complete platform. Updates deployed via `git pull && docker compose up --build -d`.

Option 2: AWS — App Runner + MongoDB Atlas

Architecture

```
┌──────────────────┐        ┌──────────────────┐
│  AWS App Runner  │        │  AWS App Runner   │
│  (Frontend)      │──────▶│  (Backend)        │
│  1 vCPU / 2GB    │ HTTPS │  1 vCPU / 2GB     │
└──────────────────┘        └──────────────────┘
                                     │
                                     │
                      ┌──────────────▼──────────────┐
                      │  MongoDB Atlas    AWS S3    │
                      │  (managed)       (existing) │
                      └──────────────────────────────┘
```

Components

Component	Service	Est. Monthly
Frontend	App Runner (1 vCPU / 2GB)	$7–15
Backend	App Runner (1 vCPU / 2GB)	$15–30
Database	MongoDB Atlas M0 (free) or M10	$0–57
File storage	AWS S3 (existing bucket)	$1–5
DNS	Route 53	$1
Email	SendGrid free tier	$0
Total		**$24–108**

Scope of Work

- Create two App Runner services (frontend, backend)
- Configure environment variables in App Runner console
- Set up MongoDB Atlas cluster, configure network access
- Configure DNS (Route 53 or external registrar)
- Set CORS origins for frontend→backend communication
- Verify health checks and auto-deploy from Git

Pros

- Zero code changes
- Git-push deploys — automatic build and release on push
- Auto-scaling built in; scales to near-zero when idle
- Managed SSL certificates
- Backend and S3 in the same AWS region — low latency for file operations
- No server maintenance or patching

Cons

- Two separate services to configure and monitor
- App Runner has a ~58% cost premium over raw EC2/Fargate per vCPU-hour
- MongoDB Atlas is a separate vendor/dashboard from AWS
- More expensive than VPS for equivalent capability at low traffic
- App Runner build times can be slow (~5–10 min per deploy)

Client Impact

No application changes. Client gets a fully managed AWS deployment with automatic scaling. Monthly cost varies with traffic — idle periods cost less, spikes cost more.

Option 3: Azure — App Service + MongoDB Atlas

Architecture

```
┌──────────────────┐        ┌──────────────────┐
│  Azure App Svc   │        │  Azure App Svc   │
│  B1 (Frontend)   │──────▶│  B2 (Backend)    │
│  1 core / 1.75GB │ HTTPS │  2 core / 3.5GB  │
└──────────────────┘        └──────────┬───────┘
                                       │
                     ┌─────────────────▼──────────────┐
                     │  MongoDB Atlas    AWS S3       │
                     │  (managed)    or Azure Blob    │
                     └────────────────────────────────┘
```

Components

Component	Service	Est. Monthly
Frontend	App Service B1 Linux (1 core / 1.75GB)	~$13
Backend	App Service B2 Linux (2 cores / 3.5GB)	~$26
Database	MongoDB Atlas M0 (free) or M10	$0–57
File storage	AWS S3 (keep existing) or Azure Blob	$1–5
DNS	Azure DNS or external	$1
Email	SendGrid free tier	$0
Total		**$40–101**

Scope of Work

- Create two App Service plans (frontend, backend)
- Configure Git deployment or container deployment
- Set all environment variables in App Service Configuration
- Set up MongoDB Atlas, whitelist Azure outbound IPs
- Configure custom domain and SSL binding
- Optionally migrate S3 storage to Azure Blob Storage
- Verify CORS, health checks, and end-to-end functionality

Pros

- Zero code changes (if keeping S3)
- App Service supports Git deploy and Docker container deploy
- Fixed monthly pricing — no usage surprises
- Azure AD integration available for enterprise clients

- MongoDB Atlas runs on Azure infrastructure (same-region option)
- Managed SSL and custom domains included

Cons

- B1 tier lacks auto-scaling (requires Standard S1+ tier at ~$55/mo)
- More expensive than VPS or Render for equivalent capability
- Azure portal is complex compared to simpler PaaS options
- If migrating S3→Blob, requires storage abstraction changes (~2 hrs)
- MongoDB Atlas is still a separate vendor from Azure

Client Impact

No application changes if keeping S3. If client requires all-Azure storage, minor configuration change to use Azure Blob Storage (existing S3 abstraction layer supports this with environment variable changes only). Suitable for enterprise clients with existing Azure commitments.

Option 4a: Vercel (Frontend) + Render (Backend)

Architecture

```
┌──────────────────┐        ┌──────────────────┐
│  Vercel          │        │  Render          │
│  (Frontend)      │──────► │  (Backend)       │
│  Edge CDN        │ HTTPS  │  Standard / 2GB  │
│  Auto-scaling    │        │                  │
└──────────────────┘        └──────────────────┘
                                     │
                                     ▼
                    ┌──────────────────────────┐
                    │  MongoDB Atlas    AWS S3 │
                    │  (managed)       (existing) │
                    └──────────────────────────┘
```

Components

Component	Service	Est. Monthly
Frontend	Vercel Free or Pro ($20)	$0–20
Backend	Render Standard (2GB RAM)	$25
Database	MongoDB Atlas M0 (free) or M10	$0–57
File storage	AWS S3	$1–5
Email	SendGrid free tier	$0
Total		**$26–107**

Scope of Work

- Connect repository to Vercel, configure build settings
- Set frontend environment variables in Vercel dashboard
- Create Render web service pointing to `/server` directory
- Set backend environment variables in Render dashboard

- Set up MongoDB Atlas cluster
- Configure CORS (Vercel domain allowed on backend)
- DNS configuration on Vercel (custom domain)
- Smoke test all routes and functionality

Pros

- Zero code changes
- Vercel is purpose-built for Next.js — edge CDN, image optimization, instant deploys
- Render is simple — connect Git repo, set env vars, done
- Frontend auto-scales globally via Vercel's edge network
- Two simple dashboards, both developer-friendly
- Vercel free tier is generous for low-traffic sites

Cons

- Two separate platforms to manage (Vercel + Render)
- Backend on Render doesn't auto-scale below Standard tier
- Cross-platform debugging requires checking two dashboards
- Render Standard tier is fixed cost regardless of traffic
- Vercel Pro required for team access and advanced features

Client Impact

No application changes. Best frontend performance of any option due to Vercel's edge network. Client gets two separate dashboards for monitoring. Recommended for clients who prioritize frontend speed and developer experience.

Option 4b: Render (Both Services)

Architecture

```
┌──────────────────┐        ┌──────────────────┐
│  Render          │        │  Render          │
│  (Frontend)      │──────► │  (Backend)       │
│  Starter / 512MB │ HTTPS  │  Standard / 2GB  │
└──────────────────┘        └──────────────────┘
                                     │
                                     │
                      ┌──────────────▼──────────────┐
                      │  MongoDB Atlas    AWS S3    │
                      │  (managed)       (existing) │
                      └──────────────────────────────┘
```

Components

Component	Service	Est. Monthly
Frontend	Render Starter (512MB)	$7
Backend	Render Standard (2GB RAM)	$25
Database	MongoDB Atlas M0 (free) or M10	$0–57
File storage	AWS S3	$1–5
Email	SendGrid free tier	$0

Total		**$33–94**

Scope of Work

- Create two Render web services (frontend, backend)
- Connect Git repository to both services
- Configure environment variables for each service
- Set up MongoDB Atlas cluster
- Configure CORS and inter-service communication
- DNS and custom domain setup
- Smoke test all functionality

Pros

- Zero code changes
- Single dashboard for both services
- Simplest managed platform — minimal configuration
- Git-push deploys for both services
- Managed SSL and custom domains
- Straightforward pricing with no usage surprises

Cons

- Frontend doesn't benefit from Next.js-specific optimizations (no edge CDN)
- No auto-scaling on Starter/Standard tiers
- Render's build times can be slower than Vercel
- Less ecosystem tooling compared to AWS or Azure
- Backend Standard tier is the minimum viable — no cheaper option

Client Impact

No application changes. Single platform, single dashboard. Simplest option for clients who want minimal operational complexity. Tradeoff is less frontend performance optimization compared to Option 4a.

Option 5: AWS Native — App Runner + DocumentDB

Architecture

```
┌──────────────────────────────────────────────┐
│                  AWS VPC                     │
│  ┌──────────────┐       ┌──────────────────┐  │
│  │ App Runner   │       │  App Runner      │  │
│  │ (Frontend)   │──────►│  (Backend)       │  │
│  │ 1 vCPU / 2GB │       │  1 vCPU / 2GB   │  │
│  └──────────────┘       └──────────┬───────┘  │
│                                    │          │
│                        ┌───────────▼──────┐   │
│                        │  DocumentDB      │   │
│                        │  db.t3.medium    │   │
│                        └──────────────────┘   │
│                                               │
│                        ┌──────────────────┐   │
│                        │  AWS S3 (existing)  │
```

```
│                                   └──────────────────────────┘   │
└──────────────────────────────────────────────────────────────────┘
```

Components

Component	Service	Est. Monthly
Frontend	App Runner (1 vCPU / 2GB)	$7–15
Backend	App Runner (1 vCPU / 2GB)	$15–30
Database	DocumentDB db.t3.medium	~$57
Storage	DocumentDB storage ($0.10/GB)	$1–3
File storage	AWS S3 (existing)	$1–5
VPC networking	Included	$0
Email	SendGrid free tier	$0
Total		**$80–107**

Scope of Work

This option requires code changes due to DocumentDB's incomplete MongoDB compatibility.

Infrastructure:

- Create VPC with subnets and security groups
- Provision DocumentDB cluster (db.t3.medium)
- Create App Runner services with VPC connector
- Configure environment variables and networking
- DNS and SSL configuration

Code Changes Required:

File	Change	Reason
server/models/index.js	Remove collation-based unique index on username; add `usernameLower` field	DocumentDB does not support collation indexes
server/models/index.js	Validate text index compatibility or replace with `$regex` fallback	DocumentDB text indexes are limited (English only, no array fields)
server/routes/auth.js	Rewrite login query to use normalized `usernameLower` field instead of `.collation()`	`.collation()` not supported; login breaks without this change
server/routes/auth.js	Rewrite setup endpoint duplicate checking	Same collation issue
server/routes/users.js	Rewrite all user lookup and uniqueness queries (4 locations)	All use `.collation()` for case-insensitive matching
server/routes/users.js	Update duplicate-check-on-update logic	Uses `$ne` + collation combination
server/index.js	Update startup user queries	Password migration lookup uses collation

New: migration script	Populate `usernameLower` field for existing users	Normalized field needs backfill

Pros

- All-AWS infrastructure — single vendor, single bill
- VPC keeps database traffic private (not over public internet)
- DocumentDB is fully managed with automated backups
- No external vendor dependency for any component
- Consistent monitoring via CloudWatch

Cons

- **Most expensive option** — DocumentDB alone is $57/mo with no free tier
- Requires auth/user code refactor (~48–64 hours including testing)
- DocumentDB has ~85% MongoDB compatibility — ongoing risk of hitting gaps
- VPC setup adds infrastructure complexity
- Nearly as much work as a full PostgreSQL rewrite (Option 7) at higher monthly cost
- Text search is limited (English only, no compound text indexes)

Client Impact

Requires code changes to authentication and user management. Application behavior is unchanged from the user's perspective, but the underlying queries are rewritten for DocumentDB compatibility. Client is locked into AWS for the database tier. **This option has the worst cost-to-value ratio** — higher monthly cost than any other option, with more work than Options 1–4b, and only marginally less work than a full PostgreSQL rewrite.

Option 6: Azure Native — App Service + CosmosDB

Architecture

```
┌──────────────────┐        ┌──────────────────┐
│  Azure App Svc   │        │  Azure App Svc   │
│  B1 (Frontend)   │──────▶│  B2 (Backend)    │
│  1 core / 1.75GB │ HTTPS │  2 core / 3.5GB  │
└──────────────────┘        └──────────────────┘
                                     │
                                     │
                     ┌───────────────▼──────────────┐
                     │  Azure CosmosDB              │
                     │  (MongoDB API, serverless)   │
                     ├──────────────────────────────┤
                     │  Azure Blob Storage          │
                     │  (replaces AWS S3)           │
                     └──────────────────────────────┘
```

Components

Component	Service	Est. Monthly
Frontend	App Service B1 Linux	~$13
Backend	App Service B2 Linux	~$26
Database	CosmosDB serverless (MongoDB API)	~$1–10

Database storage	CosmosDB ($0.25/GB)	~$1–3
File storage	Azure Blob Storage (hot tier)	$2–5
Email	SendGrid free tier	$0
Total		**$41–54**

Scope of Work

Infrastructure:

- Create App Service plans and deploy from Git
- Provision CosmosDB account with MongoDB API (serverless mode)
- Create Azure Blob Storage container
- Configure networking, environment variables, custom domains

Code Changes Required:

File	Change	Reason
server/routes/cards.js	Test and fix text search queries	CosmosDB text index behavior differs from MongoDB
server/routes/auth.js	Test collation queries; may need workarounds	CosmosDB has partial collation support (~90%)
server/routes/users.js	Test case-insensitive uniqueness	Same collation concern
server/utils/s3Storage.js	Add Azure Blob Storage driver or use S3-compatible API	Replacing AWS S3 with Azure Blob
Environment config	New Azure-specific environment variables	Blob Storage connection strings, CosmosDB URI

Pros

- All-Azure infrastructure — single vendor for enterprise clients
- CosmosDB serverless is very cheap for low-traffic sites ($1–10/mo)
- CosmosDB has a permanent free tier (1000 RU/s + 25GB)
- ~90% MongoDB compatibility — closer than DocumentDB
- Azure AD integration for enterprise SSO
- No VPC complexity — services communicate via connection strings

Cons

- Requires compatibility testing across all routes (~32–48 hours)
- CosmosDB RU-based pricing is unpredictable under traffic spikes
- S3→Blob migration adds storage abstraction work
- CosmosDB MongoDB API has known operator gaps
- B1 tier lacks auto-scaling (Standard S1 adds ~$30/mo for scaling)
- More complex than Options 1–4b with no functional benefit to the end user

Client Impact

Requires compatibility testing and potential fixes to search and authentication queries. Storage layer changes from AWS S3 to Azure Blob. Application behavior is unchanged from the user's perspective. Suitable for enterprise clients with Azure-only mandates.

Option 7: PostgreSQL Rewrite (Cloud-Agnostic)

Architecture

```
┌──────────────────┐        ┌──────────────────┐
│  Any Compute     │        │  Any Compute     │
│  (Frontend)      │──────►│  (Backend)       │
│                  │ HTTPS │                   │
└──────────────────┘        └──────────────────┘
                                     │
                                     │
                      ┌──────────────▼──────────────┐
                      │  Any PostgreSQL             │
                      │  (RDS, Azure DB, Supabase,  │
                      │   Neon, self-hosted)        │
                      ├─────────────────────────────┤
                      │  Any Object Storage         │
                      │  (AWS S3 / Azure Blob / R2) │
                      └─────────────────────────────┘
```

Components (example: VPS + Supabase)

Component	Service	Est. Monthly
Compute	Any from Options 1–4b	$13–45
Database	Supabase free / Neon free / RDS	$0–25
File storage	AWS S3 or any S3-compatible	$1–5
Email	SendGrid free tier	$0
Total		**$14–75**

Scope of Work

Full rewrite of the database layer from MongoDB/Mongoose to PostgreSQL with Knex.js query builder.

Code Changes Required:

File	Lines	Change	Complexity
server/models/index.js (170 lines)	Replace entirely	Mongoose schemas → PostgreSQL table definitions + Knex migrations	High
server/db/connection.js (28 lines)	Replace entirely	Mongoose connection → Knex/pg connection pool	Low
server/routes/cards.js (975 lines)	Rewrite all queries	25 Mongoose calls → Knex query builder; filter/sort/pagination logic	High

server/routes/auth.js (370 lines)	Rewrite all queries	Login, setup, password reset → SQL queries	Medium
server/routes/users.js (282 lines)	Rewrite all queries	User CRUD with case-insensitive uniqueness → SQL	Medium
server/utils/cardHelpers.js (544 lines)	Rewrite tag operations	Upsert/increment → `INSERT...ON CONFLICT`, `UPDATE...SET count = count + 1`	Medium
server/utils/trashCleanup.js (114 lines)	Rewrite queries	Soft delete cleanup → SQL date comparisons	Low
server/utils/healthCheck.js (268 lines)	Rewrite health probes	Mongoose connection state → pg pool health	Low
server/index.js (315 lines)	Rewrite DB init	Startup queries, seeding → SQL	Medium
server/routes/tags.js (32 lines)	Rewrite query	Single find → `SELECT...ORDER BY`	Low
server/routes/files.js (253 lines)	Rewrite lookup	Single findById → `SELECT...WHERE id =`	Low
server/routes/health.js (55 lines)	Rewrite ping	Mongoose ping → `SELECT 1`	Low
New: migration script	Data migration	MongoDB export → PostgreSQL import	Medium

Key Technical Decisions:

Decision	Recommended Approach
Query builder	Knex.js — chainable API similar to Mongoose, migration support
Tags storage	`TEXT[]` column with `ANY()` queries, or junction table
Full-text search	PostgreSQL native `tsvector` / `tsquery` on description field
Case-insensitive matching	`CITEXT` extension or `LOWER()` indexes
Soft deletes	Same pattern — `deleted_at` timestamp column with filtered queries
ID format	`UUID` primary keys (replacing MongoDB ObjectId)

Pros

- Eliminates MongoDB dependency entirely
- PostgreSQL is available free on every major platform (Supabase, Neon, Railway, Render)
- Lowest possible monthly cost — free database tiers available everywhere
- Maximum deployment portability — runs on any cloud or self-hosted
- PostgreSQL expertise is more widely available than MongoDB
- Native full-text search without external services
- Strongest long-term flexibility for future hosting changes

Cons

- Largest scope of work (~56–72 hours)
- Touches every backend file — highest risk of regression
- Requires thorough integration testing across all routes

- Tags-as-arrays queries behave differently in PostgreSQL
- MongoDB → PostgreSQL data migration adds one-time complexity
- No benefit to the end user — identical application behavior

Client Impact

No visible changes to the application. All features, UI, and behavior remain identical. The change is entirely infrastructure — MongoDB is replaced with PostgreSQL as the backing database. This investment pays off through lower hosting costs and broader deployment flexibility for all future deployments.

Recommendation

For most client deployments:

- **Option 1 (VPS + Docker)** for cost-sensitive clients who want the lowest monthly bill
- **Option 4a (Vercel + Render)** for clients who want zero server management with excellent frontend performance
- **Option 2 (AWS + Atlas)** for clients already invested in AWS infrastructure

Options 5 and 6 are only justified by a hard vendor-lock requirement. Option 7 is a strategic investment that pays off across multiple client deployments by eliminating the MongoDB dependency.